Phase 10B - phase10b_report

PHASE 10B — AI EXPLANATION LAYER

Real Estate AI Decision Intelligence Platform — Phase Completion Report

Covers Master Prompt §10.10 only, per explicit direction splitting Phase 10 into 10A (Power BI + all 7 dashboards, delivered previously) and 10B (this phase).

---

1. PHASE STATUS

PHASE 10B — COMPLETE. A real Gemini API integration, a deterministic numeric-fidelity validator, and a safe fallback mechanism are built and tested. **15/15 automated tests passing** (14 original + 1 audit-fix regression test – see Section 5B). Two real bugs were found and fixed in the validator during this phase’s own build, before any explanation was shipped.

---

2. OBJECTIVE

Build the actual LLM-based explanation layer §10.10 describes — reading a Phase 9 `DecisionResult` and explaining it in business language — while enforcing, in code rather than by trusting the prompt, that “the LLM must preserve the actual underlying values… it cannot change IRR, NPV, ROI, risk score, decision score, decision state.”

---

3. WHY THIS IS DIFFERENT FROM PHASE 9’S EXPLANATION AGENT

Phase 9’s `explanation_agent.py` is a deterministic template — explicitly not an LLM call, by design, to keep that phase’s decision-critical path and test suite free of any LLM dependency. Phase 10B is where the platform’s actual LLM explanation belongs, and it is built with a hard governance mechanism specifically because an LLM call is now in the loop:

Every LLM response is validated before a person ever sees it. `numeric_fidelity_validator.py` extracts every protected number (IRR, NPV, cap rate, ROI, DSCR, decision score, decision confidence, probability of negative NPV, probability of meeting target IRR, location score, market strength score) directly from the structured `DecisionResult` — never from the LLM's text — and checks whether each one the explanation actually mentions was reproduced correctly. A protected number that appears **altered** is a hard failure. The LLM's text is then **discarded entirely**, not edited or partially trusted, and the caller receives Phase 9's deterministic template instead, with a clear, logged reason why.

---

4. NO LIVE API KEY WAS AVAILABLE IN THIS BUILD SANDBOX

Stated plainly, same honesty standard as "no live PostgreSQL" in every prior phase: this build environment has no `GEMINI_API_KEY` configured. `ai_explanation_layer.py` is written as a real, production-ready integration (`urllib` POST to `https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent`, correct headers, correct message format, `gemini-2.5-flash` (configurable via GEMINI_MODEL)), but no live call was actually made to produce this deliverable's examples. Instead:

- **The no-key path was exercised for real** — `generate_ai_explanation()` correctly detects the missing key and falls back to Phase 9's template immediately, without attempting a network call, tested by `test_no_api_key_falls_back_immediately`.
- **The LLM-response-handling path was tested with `unittest.mock.patch`** on `_call_gemini()`, substituting synthetic (faithful and deliberately tampered) response text. This is disclosed directly in `test_phase10b.py`'s own docstring — it is not presented as having exercised the real network call.
- **`examples/example_SIMULATED_llm_response_property_2731.txt`** is explicitly labeled, in its own first line, as simulated and not a real API response — used only to demonstrate the fidelity-validation mechanism end-to-end with a plausible, hand-written business explanation.

Anyone running this deliverable with a real `GEMINI_API_KEY` set will exercise the actual `_call_gemini()` code path with no code changes required.

---

5. TWO REAL BUGS FOUND AND FIXED IN THE VALIDATOR ITSELF

Testing the validator honestly — including deliberately trying to break it with a tampered example — surfaced two real defects before this phase shipped:

Bug 1 — sentence-period consumed as a decimal point. The first numeric-extraction regex (`-?\d[\d,]*\.?\d*`) greedily matched a sentence-ending period as if it were a decimal separator, turning `"NPV is ₹2,847,332."` into the token `"2847332."` (trailing dot), which then never matched the correct representation `"2847332"`. **Fixed** by requiring at least one digit after any decimal point (`-?\d[\d,]*(?:\.\d+)?`), confirmed by `test_numbers_in_text_does_not_swallow_sentence_period`.

Bug 2 — label-format mismatch let a tampered decision_score through undetected. The validator's fallback logic (when a number's exact form isn't found) checks whether the field's *label* is discussed nearby to decide "altered" vs. "just omitted." The label for `decision_score` was originally the raw field name `"decision_score"` (with an underscore), but natural prose writes "Decision score" (with a space) — so a genuinely tampered decision score (92.0 instead of the real 69.2) was being classified as merely *omitted*, not *contradicted*, and would have passed validation. **Fixed** by using human-readable labels ("decision score", "decision confidence"), confirmed by `test_altered_decision_score_is_caught`, a dedicated regression test.

Both bugs were found by adversarial self-testing — deliberately constructing a tampered example and checking whether the validator caught it — which is exactly the test discipline this kind of governance mechanism requires. A validator that hasn't been tested against a deliberately-broken input hasn't really been tested.

5B. POST-DELIVERY AUDIT: PROVIDER SWITCHED FROM ANTHROPIC

TO GEMINI, PLUS AN INHERITED ROI BUG FIXED

This phase was originally built against Anthropic's Messages API. Per explicit direction, it has been migrated to **Google Gemini** (`generateContent` REST endpoint) instead:

- `_call_anthropic()` → `_call_gemini()`, now posting to `https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent` with the `x-goog-api-key` header (Gemini's auth convention) instead of Anthropic's `x-api-key` + `anthropic-version` headers, and Gemini's `contents` / `parts` / `system_instruction` request shape instead of Anthropic's `system` / `messages` shape.
- Environment variable renamed `ANTHROPIC_API_KEY` → `GEMINI_API_KEY`; model name is now configurable via `GEMINI_MODEL` (default `gemini-2.5-flash`) rather than hardcoded, since Google's model lineup changes on its own schedule.
- **`numeric_fidelity_validator.py` required NO changes at all** — it validates the LLM's output *text* against the *DecisionResult*, and has no awareness of which provider produced that text. This is exactly the intended separation of concerns (Section 3.4: the governance mechanism must not be coupled to a specific LLM implementation).
- `test_phase10b.py`: `unittest.mock.patch` targets updated from `_call_anthropic` to `_call_gemini`; the no-key-path assertion updated to check for `GEMINI_API_KEY`. One new test was added: `test_real_result_roi_protected_number_is_net_return_not_gross_multiple`.

That last new test exists because the same independent review that migrated the provider also found that **`src/finance/engine.py` here was pulled in from Phase 6 *before* Phase 6's own post-delivery audit fixed a bug in `roi_total_holding_period`** (it was computing a gross cash multiple instead of a net return, overstating ROI by exactly 100 percentage points in every case — the same inherited-stale-copy issue independently found in Phases 7, 9, and 10A). This matters more here than in Phase 9: `numeric_fidelity_validator.py` explicitly lists ROI as a **protected number** (per Master Prompt §10.10), meaning it would have faithfully *defended* the wrong value against correction by any LLM that happened to state the (correct) real-world figure — a validator protecting a bug is worse than no validator. **Fixed** by re-copying Phase 6's corrected `engine.py` (re-verified byte-identical via `diff`). All 15 tests (14 original + 1 new) pass; the example files in `examples/` were re-checked and do not state a raw ROI figure, so none needed correction beyond the Anthropic→Gemini text updates.

5C. POST-DELIVERY REBUILD: INTERACTIVE CHAT CLI (`chat.py`)

A user-supplied `chat.py` draft added a terminal Q&A loop over this phase, but a review of it found it could reliably do only one thing — try to resolve *some* property id and explain it — and did that unreliably:

- It matched *any* bare 4-digit number in the question as a `property_id` (`re.search(r'\b\d{4}\b', ...)`), which misfires on a year, a percentage, or a locality number quoted in the same sentence.
- "Top 5" / "best investment" questions returned `df['property_id'].head(5)` — literally the first 5 rows of the raw CSV file, which has no relationship whatsoever to investment quality. A "best properties" answer was, in effect, random.
- There was no path at all for a general/conceptual question ("why were some models rejected?", "what does UNCALIBRATED mean?") — anything that didn't match a property/locality pattern silently fell through to "pick the first property in the file" (`df.iloc[0]`), which is a confident-looking wrong answer to a question that was never about a property in the first place.
- It read `result.get('valid')` from `generate_ai_explanation()`'s return value — that key does not exist (the actual keys are `explanation`, `source`, `fidelity_report`, `fallback_reason`), so every response silently printed `VALID: None`.

Rebuilt as a proper query-routing pipeline rather than a single regex:

Module	Responsibility
`src/chat/query_router.py`	Classifies any question into PROPERTY / COMPARE / LOCALITY / RANKING / GENERAL / UNCLEAR *before* any pipeline or LLM call is made. Refuses to guess — an unrecognized question returns `UNCLEAR` with example phrasings, rather than silently defaulting to some property

	(the same “no silent guessing” principle used throughout this platform, Master Prompt §3.2).
`src/chat/ranking.py`	Answers “top N / best / safest / highest-return” by actually running the deterministic decision pipeline across Phase 6’s `select_demo_sample` and sorting by the requested metric — never by file order.
`src/chat/locality_lookup.py`	Resolves a locality by numeric id **or by name** (exact, partial, or fuzzy match against `localities.csv`) — the original only accepted a bare id.
`src/chat/general_qa.py`	Answers conceptual questions, grounded in a hand-written `platform_knowledge.py` summary of this project’s own architecture, formulas, and model statuses — with a full-text fallback (not a generic “I don’t know”) when no API key is set.
`src/explanation/ai_explanation_layer.py`	Refactored to expose a generic `_call_gemini_raw(system_prompt, user_message, api_key)`, so `general_qa.py` reuses the exact same Gemini call path as property explanations — there is exactly one place in the codebase that talks to the Gemini API.

Two real bugs found and fixed during this rebuild’s own testing (not just claimed — reproduced and corrected):

1. **Module name collision.** `src/finance/batch_runner.py` and `src/geo/batch_runner.py` share a filename; a plain `import batch_runner` is ambiguous and picked up the wrong one (geo’s, which has no `select_demo_sample`) depending on `sys.path` order. Fixed by loading the finance module from its exact file path via `importlib.util`, bypassing `sys.path` resolution entirely — `src/chat/ranking.py`.
2. **“Safest investment” surfaced an AVOID property with a negative IRR** ahead of genuine INVEST properties, because sorting purely by `decision_confidence` doesn’t account for whether the underlying decision was actually good — an AVOID property can coincidentally have high confidence in *being told to avoid it*. Fixed by making decision quality (INVEST > HOLD > AVOID) the primary sort key and the requested metric the secondary key; locked in by `test_ranking_prioritizes_decision_quality_over_raw_metric` in `test_chat.py`.

Testing: `test_chat.py` (new, 29 tests: router classification for every intent, ranking correctness and the tie-breaking fix, locality resolution by id/exact-name/partial-name/unknown-name, and 6 end-to-end `answer()` calls) — all passing. The original `src/explanation/test_phase10b.py` suite (15 tests) was re-run after the `ai_explanation_layer.py` refactor and confirmed unaffected.

---

6. TESTS EXECUTED / PASSED / FAILED

15/15 passed (14 original + 1 added during the post-delivery audit – see Section 5B). Coverage: the regex regression test above, protected-number extraction correctly skipping absent fields, a faithful explanation passing validation end-to-end, an altered IRR being caught, the decision-score label-mismatch regression test, an altered decision *state* (INVEST rewritten as HOLD with otherwise-correct numbers) correctly failing even though every number matched, an omitted number correctly NOT being treated as a failure (the LLM may reasonably not mention everything), lakh/crore currency representations being accepted as valid restatements of the same NPV, the no-API-key path falling back immediately, a mocked faithful LLM response being returned as-is, a mocked tampered LLM response being discarded (with the tampered number confirmed absent from what the user actually sees), a mocked response that tries to override the decision state being discarded in favor of the correct AVOID decision, a simulated network failure falling back gracefully, and confirmation that the fallback path never returns an empty explanation for either an INVEST or an AVOID case.

`test_chat.py` – 29/29 passed (added during the Section 5C chat-CLI rebuild). Coverage: router classification for all six intents (property/compare/locality/ranking/general/unclear) across 14 test cases including edge cases (a stray “compare” with only one number, an empty query), ranking correctness (requested count, sort order within a decision tier, INSUFFICIENT properties excluded, and the decision-

quality-first tie-breaking regression test), locality resolution by id/exact-name/partial-name/unknown-name, and 6 end-to-end `answer()` calls covering a valid property, a nonexistent property (must degrade to INSUFFICIENT, not crash), a ranking query, a locality query, an unclear query, and a comparison query.

---

7. MASTER PROMPT REQUIREMENTS COVERAGE

Section	Requirement	Status
10.10	LLM receives DecisionResult and explains it	✅ `ai_explanation_layer.generate_ai_explanation`
10.10	May explain: why, positive/negative factors, financial, risk, location, evidence, conditions, actions	✅ enforced in `SYSTEM_PROMPT`
10.10	Must preserve IRR/NPV/ROI/risk score/decision score/decision state	✅ enforced in code by `numeric_fidelity_validator.py`, not just the prompt
3.5	LLM must not modify calculations or override decision policy	✅ tampered/overriding responses are discarded entirely, never partially trusted
3.2	Never guess when data is unavailable	✅ no-API-key path falls back rather than fabricating an explanation

---

8. BUSINESS VALUE

- A real, usable LLM explanation layer that a business user would actually want to read — plain language, organized sections — sitting on top of every deterministic number the platform already computed and tested across Phases 6-9.
- The fidelity validator is the difference between “an LLM explains our numbers” and “an LLM explains our numbers, and we have proof it didn’t change any of them.” Every explanation this module returns as `source: "llm"` carries a `fidelity_report` showing exactly which protected numbers were checked and matched — an auditable record, not a trust-me claim.
- The fallback guarantees the platform never goes silent: even total API failure, missing credentials, or a badly-behaved model response still produces a correct, complete explanation via Phase 9’s template.

---

9. KNOWN LIMITATIONS (see also `docs/limitations.md`)

1. No live Gemini API call was made to produce this deliverable — no `GEMINI_API_KEY` was available in this build sandbox. The integration code is real and production-ready; end-to-end network behavior should be confirmed once a key is available.
2. The numeric fidelity validator checks 10 specific protected fields (§10.10’s explicit list plus the decision policy’s own component scores) — it does not attempt to validate every number an LLM might state (e.g., a locality’s population density, if the LLM chose to mention it). Expanding `PROTECTED_FIELD_PATHS` is straightforward if more fields need this guarantee.
3. The validator’s “contradicted vs. omitted” classification depends on the field’s label appearing in the explanation text in recognizable form — an LLM response that discusses a concept using entirely different wording than the labels in `PROTECTED_FIELD_PATHS` (e.g., writing “the return” instead of “IRR”) could theoretically have an altered number classified as omitted rather than contradicted. The system prompt instructs the LLM to use standard terms, but this is a prompt-level mitigation, not a code-level guarantee, and is disclosed here rather than assumed away.
4. Currency representations accept whole-rupee, lakh, crore, and million forms — but not every possible human phrasing (e.g., “two point eight million” in words). A correct number stated in an unrecognized format could be misclassified as omitted.
5. The system prompt is a governance control, not a guarantee by itself — this is precisely why the

code-level validator exists as the actual enforcement mechanism, per Section 3 above.

---

10. DEPENDENCIES

Phase 9 (`orchestrator.run_decision_pipeline`, reused unchanged, for producing the `DecisionResult` objects this phase explains) and Phase 9's `explanation_agent.py` (reused unchanged, as the fallback).

---

11. SECURITY CONSIDERATIONS

`GEMINI_API_KEY` is read from an environment variable only, never hardcoded, matching the `DB_USER` / `DB_PASSWORD` convention used throughout this platform. **Post-delivery addition:** a `.env.example` template and `.gitignore` entry were added so every credential (DB and Gemini) is set once in a local `.env` file rather than via manual `export` commands or, worse, hardcoded values — `db/connection.py` and `src/explanation/ai_explanation_layer.py` both load it automatically via `python-dotenv` at import time. The real `.env` is never committed; only the secret-free `.env.example` template is. The API call sends the full `DecisionResult` JSON to Google's Gemini API — this includes property-level financial figures; anyone deploying this in production should confirm this data-sharing is acceptable under their own data-handling policies before enabling the live LLM path.

---

12. PERFORMANCE CONSIDERATIONS

Not measured — no live API call was made. A real deployment should expect standard Gemini API latency (typically low single-digit seconds for a ~1,200-token response) plus the fidelity validation step, which is pure Python string/dict operations and negligible by comparison.

---

13. PRODUCTION READINESS

Per Section 24: **Production-Oriented Prototype.** The integration code, governance mechanism, and fallback logic are complete and tested (with the explicit caveat in Section 4 about no live API call having been made). Recommended before relying on this in production: (a) set a real `GEMINI_API_KEY` and run the full pipeline end-to-end at least once to confirm live behavior matches the mocked-test expectations; (b) review Section 11's data-sharing consideration with the business; (c) consider expanding `PROTECTED_FIELD_PATHS` if additional fields need the same hard guarantee.

---

14. FINAL STATUS

PHASE 10B: COMPLETE. The real AI Explanation Layer is built: a genuine Gemini API integration, a deterministic numeric-fidelity validator that discards (not edits) any LLM response that alters a protected number or the decision state, and a safe fallback to Phase 9's template for every failure mode (no key, network failure, failed validation). 15/15 tests passing (including 1 added during a post-delivery audit that migrated the provider to Gemini and caught an inherited Phase 6 ROI bug – see Section 5B), two real validator bugs found and fixed through deliberate adversarial testing before shipping. **This completes all ten phases of the Real Estate AI Decision Intelligence Platform.**